

ПЛОВДИВСКИ УНИВЕРСИТЕТ
„ПАИСИЙ ХИЛЕНДАРСКИ“

Факултет по Математика и Информатика
Специалност „Софтуерно инженерство и технологии“

Курсова работа по
„Инженеринг на данни“

Тема

DataHub

Студент:

Емил Григоров Медаров

Ф№: 2501777012

1.Каква е идеята на технологията DataHub и за какво се използва?

DataHub е платформа с отворен код за метаданни, която цели да подпомогне управлението, откриването и разбирането на данни в големи и сложни екосистеми. Тя е създадена първоначално в LinkedIn, а по-късно е дарена на Linux Foundation. Основната идея на DataHub е да осигури централизиран „каталог“ и „мозък“ за всички данни в дадена организация, като свързва техническа информация за данните (например схеми, собственици и произход) с бизнес контекст (като термини от речници и документация).

На практика, DataHub се използва за:

- Откриване на данни (Data Discovery): Позволява на инженери, анализатори и учени по данни бързо да намират необходимите им набори от данни чрез мощно търсене и филтриране .
- Проследяване на произход (Data Lineage): Показва визуално "живота" на данните - откъде идват, как се трансформират и кои системи зависят от тях .
- Управление и наблюдение (Data Governance & Observability): Предоставя инструменти за управление на собствениците на данни, тагове, бизнес термин и политики за достъп, което повишава доверието в данните .

DataHub не е просто поредният инструмент, а ключов компонент на съвременния технологичен стек за данни, често определян като контекстен слой за изкуствен интелект и анализи. Той предоставя на AI системите и на хората структурирана бизнес информация, което превръща суровите данни в надеждни активи.

2.Къде най-добре пасва в data pipeline схемата?

DataHub не замества нито един от съществуващите компоненти на pipeline-а (като бази данни, ETL инструменти или BI платформи), а функционира като хоризонтален слой, който ги обединява и добавя контекст върху тях.

- Място в архитектурата: DataHub се позиционира като "мета-слой" над всички останали компоненти. Той консумира метаданни от източниците на данни (напр. Snowflake, PostgreSQL), трансформациите (dbt, Airflow), складовете и BI инструментите (Looker, Tableau) .
- Интеграция в процеса: Представете си pipeline, в който:
 1. Източник: MySQL база данни, от която се четат данни.
 2. Трансформация: dbt модел, който трансформира данните в Snowflake.

3. Визуализация: Dashboard в Looker.

В този сценарий DataHub се "вклинява" между тези стъпки, като използва recipe-та за всяка технология :

- Той ще изтегли схемата и собственика на таблицата от MySQL.
- Той ще прочете dbt манифеста, за да разбере какъв SQL се изпълнява.
- Той ще види в Airflow кога задачата е била изпълнена.
- След това, DataHub свързва всичко това в граф на произхода, показвайки, че един report в Looker зависи от dbt модел, който зависи от таблица в MySQL .

DataHub поддържа както pull модел (периодично извличане на метаданни), така и push модел (изпращане на метаданни чрез SDK или API). Вторият модел позволява актуализации в реално време, което е ключово предимство пред по-старите алтернативи .

3.Какви проблеми решава?

DataHub адресира фундаментални проблеми, които възникват, когато една компания разраства своята data платформа:

1. "Тъмните данни" (Data Darkness): В големите компании хората не знаят какви данни съществуват и къде да ги намерят. Екипът на Slack споделя, че са се борили цели 6 години с този проблем, докато не внедряват DataHub .
2. Липса на видимост на зависимости (Impact Analysis): Когато се промени структурата на критична таблица, е трудно да се разбере кои даунстрийм отчети или ML модели ще се счупят. DataHub решава това чрез end-to-end lineage, което позволява "анализ на въздействието" с няколко кликания .
3. Неефективна комуникация: Вместо да задават въпроси в Slack канали или да четат остаряла документация, потребителите могат да видят директно в DataHub кой е собственикът на даден набор от данни .
4. Бавна организация и управление в data mesh архитектури: Помага за децентрализираното управление на данни (federated governance), позволявайки на различни домейни да управляват своите собствени метаданни чрез познати процеси и CI/CD .
5. Липса на контекст за AI системи: С навлизането на AI, моделите (LLM) се нуждаят от достоверен контекст, за да дават точни отговори. DataHub играе ролята на този "контекстен слой", осигурявайки на AI агентите надеждна бизнес дефиниция и авторитетни източници .

4.Кои са основните му алтернативи/конкуренти?

Екосистемата за управление на метаданни (Data Catalogs) е разнообразна. Конкурентите могат да бъдат разделени на отворени и собственически (commercial) решения.

Категория	Инструмент	Предимства	Недостатъци спрямо DataHub
Open-Source (Основни конкуренти)	Amundsen (Lyft)	Силен фокус върху лесно търсене и откриване на данни. Добър UI/UX.	По-трудно мащабиране; няма поддръжка за push архитектура (разчита на партидно индексване) и по-ограничена поддръжка на lineage .
	OpenMetadata	Модерен стек, много бързо развиваща се общност. Набляга на data quality и документация.	Въпреки че напредва бързо, DataHub има pre-built интеграции за повече източници и по-зряла поддръжка на lineage .
	Apache Atlas	Стандарт в Hadoop екосистемата, добър за управление на данни (data governance).	Ограничена поддръжка за real-time; по-тежък и сложен за настройка в съвременен cloud-native Data Stack .
Proprietary (Enterprise конкуренти)	Alation	Пионер в областта, изкуствен интелект за автоматично тагване, силен партньорски екосистем.	Скъп и "черна кутия" (Black box). Труден за пълна персонализация и интеграция с вътрешни собствени системи .
	Collibra	Мощен фокус върху data governance (съответствие, GDPR, бизнес глосар).	Често е твърде сложен и "бюрократичен" за case-ове на чисто инженерно търсене и lineage .

Таблица 1. Основни алтернативи на DataHub

Ключови предимства на DataHub пред алтернативите:

- Push & Pull архитектура: Възможността за предаване на метаданни в реално време (реална "жива" карта) е огромно предимство пред конкуренти като Amundsen, които разчитат на периодични скриптове .
- Гъвкавост и разширяемост: DataHub позволява лесно дефиниране на собствен модел на метаданните чрез аспекти. Ако ти трябва да проследяваш специфична пропърти за твой инструмент, можеш да я добавиш без да чакаш ъпдейт от разработчиците .
- Силен lineage: От личния ми опит, DataHub предлага една от най-добрите визуализации на lineage (вкл. на ниво колона) сред решенията с отворен код .
- Интеграции: Реално поддържа над 50+ интеграции (от Snowflake до dbt и PowerBI), което покрива нуждите на 90% от съвременните екипи .

5.Примери за употреба на технологията

Ето два реалистични сценария, които илюстрират силата на DataHub в ежедневната работа.

Пример 1: Анализ на въздействие (Impact Analysis) във Funding Circle

Ситуация: Екипът на платформата за кредитиране Funding Circle трябва да промени името на колона `customer_id` на `client_id` в основната си таблица `loans` в Snowflake .

Как DataHub помага:

1. Визуализация: Инженерът отива в DataHub и търси таблицата `loans`.
2. Lineage: Отваря таба Lineage. Там той вижда граф, показващ как потокът от данни тръгва от `loans`, преминава през dbt модел `monthly_summary`, стига до Airflow DAG `financial_reports` и накрая завършва като `dashboard` в Looker, който се ползва от мениджмънта.
3. Действие: Преди да направи промяната, инженерът може да напише в коментар: "Внимание, ще променя `customer_id`", като тагне собственика на Looker dashboard-а, за да може той да си подготви отчета навреме.

Пример 2: Автоматизирано управление на метаданни (CI/CD за метаданни)

Ситуация: Екипът иска да стандартизира PII (лични данни) документацията. Всеки нов pipeline трябва да има таг "PII" и документиран собственик.

Как DataHub помага (както правят във Funding Circle):

1. Код вместо UI: Вместо да се логва в портал и да попълва форми, инженерът създава YAML файл (напр. `metadata/loans.yaml`), в който пише собственика (`owner: "finance-team"`) и таговете (`tags: ["PII", "internal"]`).
2. CI/CD интеграция: Този YAML файл се `commit`-ва в Git.
3. Автоматизация: CI pipeline-а (например Drone или GitHub Actions) използва DataHub Python SDK, за да изпрати този YAML към DataHub API.
4. Резултат: DataHub автоматично обновява метаданните. Ако някой се опита да пусне таблица без тази информация, CI pipeline-а може да го отхвърли. Така документацията става част от процеса на разработка, а не допълнителна тежест .

Технически пример: Интеграция с Glue и Spark

В Amazon Web Services, инженерите могат да конфигурират DataHub да слуша техните Spark jobs. Чрез добавяне на малък JAR файл (`acryl-spark-lineage`) към задачата, DataHub автоматично ще улови, че задачата чете от S3 bucket `raw-users` и записва в `processed-users`, като създаде `column-level lineage`. Това се случва без да се пипа бизнес логиката на самото приложение .